Cuestionario Diagnóstico

Actividad 3.2 – Contextualización e identificación de conocimientos necesarios
Competencia: Diseñar la solución de software de acuerdo con procedimientos y requisitos técnicos.
Resultado de Aprendizaje: Elaborar los artefactos de diseño del software siguiendo las prácticas de la metodología seleccionada.

NOMBRE COMPLETO DEL APRENDIZ	NÚMERO DE FICHA
Mateo Henao Lopez	3410390
GRUPO DE TRABAJO	**FECHA**
Grupo 2	22/10/2026

Instrucciones: Responda individualmente este cuestionario. No se evalúa con nota; su propósito es identificar saberes previos antes de iniciar el módulo de diseño de software. El instructor retroalimentará los resultados en la sesión de contextualización. Tiempo estimado: 25 minutos.

A. Conceptos básicos de diseño de software

1 **¿Cuál es la principal diferencia entre el análisis de requisitos y el diseño de software?**

- ◉ a) El análisis define el "qué" debe hacer el sistema; el diseño define el "cómo" se implementará.
- ○ b) El análisis se realiza después de codificar; el diseño antes.
- ○ c) Son sinónimos: ambos producen código fuente.
- ○ d) El diseño solo aplica a bases de datos; el análisis a interfaces.

2 **En diseño orientado a objetos, el principio de *alta cohesión* implica que:**

- ○ a) Una clase debe tener muchas responsabilidades para ser reutilizable.
- ◉ b) Los elementos de una clase están fuertemente relacionados y tienen un propósito único y bien definido.
- ○ c) Las clases deben depender de muchas otras clases.
- ○ d) El software debe tener el menor número de clases posible.

3 **El principio SOLID que establece que una clase debe tener una sola razón para cambiar se denomina:**

- ○ a) Open/Closed Principle (OCP)
- ○ b) Liskov Substitution Principle (LSP)
- ◉ c) Single Responsibility Principle (SRP)
- ○ d) Dependency Inversion Principle (DIP)

4 **El *bajo acoplamiento* entre módulos busca principalmente:**

- ○ a) Que los módulos compartan la mayor cantidad de variables globales.
- ○ b) Reducir las dependencias entre módulos para facilitar el mantenimiento y la evolución del software.
- ◉ c) Aumentar el número de llamadas entre clases.
- ○ d) Que todos los módulos dependan de una única clase central.

B. UML y artefactos de modelado

5 **¿Qué diagrama UML muestra la estructura estática del sistema con clases, atributos, métodos y relaciones?**

- a) Diagrama de secuencia
- b) Diagrama de casos de uso
- c) Diagrama de clases
- d) Diagrama de actividades

6 **El diagrama de despliegue UML se utiliza para:**

- a) Mostrar la interacción temporal entre objetos.
- b) Representar la configuración física de hardware y la distribución de componentes de software.
- c) Modelar los requisitos funcionales del cliente.
- d) Describir el modelo lógico de la base de datos.

7 **¿Cuál de los siguientes es un artefacto típico de la fase de diseño y NO del análisis?**

- a) Documento de visión del producto
- b) Diagrama de clases detallado con atributos y métodos
- c) Lista de stakeholders
- d) Acta de reunión con el cliente

8 **En un diagrama de clases, la multiplicidad "1..*" en una relación indica que:**

- ○ a) Existe exactamente un objeto en ese extremo.
- ○ b) Puede haber cero o muchos objetos en ese extremo.
- ◉ c) Debe haber al menos un objeto y pueden existir varios en ese extremo.
- ○ d) No se permite ningún objeto en ese extremo.

C. Patrones de diseño

9 **El patrón Singleton pertenece a la categoría de patrones:**

- ○ a) Estructurales
- ○ b) Comportamentales
- ◉ c) Creacionales
- ○ d) Arquitectónicos

10 **¿Qué patrón de diseño se utiliza típicamente para definir una familia de algoritmos intercambiables y permitir que el cliente elija uno en tiempo de ejecución?**

- ○ a) Facade
- ○ b) Strategy
- ◉ c) Adapter
- ○ d) Observer

11 **El patrón MVC (Modelo-Vista-Controlador) es un ejemplo de patrón:**

- ○ a) Creacional GoF
- ◉ b) Arquitectónico o de organización de la aplicación
- ○ c) De base de datos relacional
- ○ d) De pruebas unitarias

D. Arquitectura de software

12 **La arquitectura de software en capas se caracteriza por:**

- ◉ a) Organizar el sistema en niveles jerárquicos donde cada capa se apoya en la inferior.
- ○ b) Ejecutar todo el sistema en un único archivo monolítico sin separación.
- ○ c) Duplicar la lógica de negocio en cada capa para mayor velocidad.
- ○ d) Eliminar la separación entre interfaz de usuario y datos.

13 **¿Qué tipo de arquitectura divide la aplicación en servicios pequeños, independientes y desplegables de forma autónoma?**

- ○ a) Monolítica
- ○ b) Cliente-servidor de dos capas
- ◉ c) Microservicios
- ○ d) Batch

14 **En el contexto del proyecto formativo, ¿por qué es importante documentar la arquitectura antes de codificar?**

- ○ a) Porque es un requisito decorativo del informe final.
- ◉ b) Porque facilita la comunicación del equipo, guía la construcción y permite evaluar riesgos técnicos tempranos.
- ○ c) Porque reemplaza la necesidad de hacer pruebas.
- ○ d) Porque lo exige el cliente sin importar el contexto.

E. Metodologías de desarrollo

15 **¿Cuál de las siguientes es una metodología ágil de desarrollo de software?**

- ○ a) RUP (Rational Unified Process)
- ○ b) Cascada
- ◉ c) Scrum
- ○ d) Modelo en V

16 **En Scrum, el artefacto que contiene la lista priorizada de funcionalidades del producto se denomina:**

- ○ a) Sprint Backlog
- ◉ b) Product Backlog
- ○ c) Burndown Chart
- ○ d) Definition of Done

17 **RUP organiza el ciclo de vida en cuatro fases. ¿Cuál de estas NO es una fase de RUP?**

- ○ a) Inicio
- ○ b) Elaboración
- ○ c) Construcción
- ◉ d) Comercialización

18 **¿Qué metodología se caracteriza por iteraciones cortas, entregas frecuentes y colaboración estrecha con el cliente?**

- ○ a) Cascada pura
- ◉ b) Ágiles (Scrum, XP)
- ○ c) Modelo espiral exclusivamente
- ○ d) Desarrollo basado exclusivamente en documentación

F. Verdadero o Falso

19 **Un diseño de software bien elaborado garantiza que el software sea mantenible y escalable.**

◉ **Verdadero** ○ **Falso**

20 **El diagrama de casos de uso es un artefacto de diseño estático que muestra clases y atributos.**

○ **Verdadero** ◉ **Falso**

21 **Los patrones de diseño GoF se clasifican en creacionales, estructurales y comportamentales.**

(●) Verdadero ( ) Falso

22 **La arquitectura de microservicios implica desplegar toda la aplicación como un único bloque monolítico.**

( ) Verdadero (●) Falso

G. Pregunta abierta

23 **Mencione al menos tres artefactos de diseño que consideraría indispensables antes de escribir la primera línea de código en un proyecto de software, y explique brevemente por qué.**

previniendo rediseños costosos en la base de datos.

3. Wireframes o Prototipos: Muestran visualmente las pantallas y flujos de la interfaz para validar la experiencia con los usuarios antes de escribir código.

Retroalimentación del instructor

Este apartado lo diligencia el instructor durante la sesión de socialización de resultados del cuestionario diagnóstico (punto 3.2).

Grupo	Tema asignado (Taller de análisis)	Aciertos	Desaciertos	Observaciones / conceptos a reforzar
Grupo 1	Tema del taller	N°	N°	Ej. Reforzar diagrama de despl
Grupo 2	Tema del taller	Buena prese	Salieron a le	Aprenderse la presentacion, nc
Grupo 3	Tema del taller	N°	N°	...

Nota para el instructor: Recuerde que en esta etapa (3.2) no se recolectan evidencias formales para juicio evaluativo; los resultados del cuestionario son insumos para la nivelación de conocimientos y para orientar la retroalimentación. Al finalizar, socialice las respuestas correctas y registre las observaciones en la tabla anterior, articulándolas con los informes del "Taller de análisis" que cada grupo expondrá.

Elaborado por: Jose German Estrada Clavijo — Instructor Técnico ADSO
Guía asociada: GFPI-F-135 Versión 4 — Punto 3.2 Actividades de contextualización
Ambiente: SISTEMAS 1 · **Fecha:** 01 de octubre de 2026

© Servicio Nacional de Aprendizaje (SENA) - Formación Profesional Integral · Formato GFPI-F-135 Versión 4